回顾：ISA 与微架构

ISA 规定程序可见的操作与行为

微架构在设计约束下实现这些规定

ISA：程序可见的规定	微架构：具体实现
ADD 的操作编码	寄存器文件的读写端口数
通用寄存器的数量	乘法指令需要的时钟周期数
寻址与数据表示规则	是否采用流水线

同一套 ISA 可以对应不同的处理器实现

回顾：指令编码与操作数组织

操作码说明做什么，操作数信息指出数据及结果的位置

示意汇编：ADD R1, R2, R3

输入：R2、R3　　结果：R1 = R2 + R3

组织方式	加法的操作数与结果
0 地址：堆栈	栈顶两个数运算，结果留在栈顶
1 地址：累加器	一个显式操作数，累加器兼作输入和结果
2 地址	两个显式位置，结果覆盖其中一个输入
3 地址	分别指定两个输入和一个结果位置

“地址数”指显式指定的操作数位置数量

减少显式操作数可节省编码位

但取数、搬移或重排可能需要额外指令

回顾：寻址与数据表示

怎样找到数据

按字节编址，数组起址 1000，每个元素占 4 字节

A[0]	A[1]	A[2]
1000	1004	1008

A[2] 的地址 = 1000 + 2 × 4 = 1008

怎样解释数据

同一个字节

01000001

解释规则	含义
无符号整数	65
ASCII 字符	A

地址帮助定位数据，解释规则决定这些位的含义

回顾：专门指令与软硬件分工

同一项双向链表插入，可以有不同的实现方式

ISA 提供专门指令	软件组合基本指令
一条指令规定一组链接修改 要求遵守指定的数据布局	读取、写入、地址计算 组合完成同一项插入任务

可能的收益：代码更紧凑，减少显式指令条数

需要付出的代价：实现复杂度与数据布局约束

设计选择要同时考虑
代码大小、执行代价和硬件成本

今天的内容

- 指令集架构的设计权衡(继续)
- MIPS指令集简介

指令集架构（ISA）的组成元素

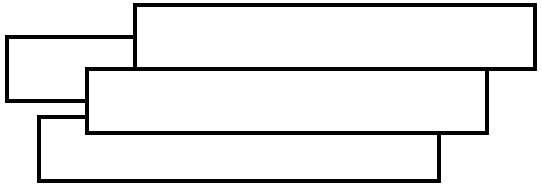
- **寄存器（Registers）**
  - 数量（How many）
  - 每个寄存器的大小（Size of each register）

- **为什么寄存器是一个好主意？（Why is having registers a good idea?）**
  - 因为程序表现出一种称为局部性（data locality） 的特性。
  - 最近生成或访问过的值很可能会被再次使用（时间局部性，temporal locality）。
    - 将该值存储在寄存器中，可以避免每次需要使用时都去访问内存。

程序员可见的状态

M[0]
M[1]
M[2]
M[3]
M[4]
M[N-1]

内存（Memory）
一组由地址索引的存储位置组成的数组。



寄存器（Registers）
在 ISA（指令集体系结构）中被赋予
定名称（而非地址）。
分为 通用寄存器（general purpose
和 专用寄存器（special purpose）

程序计数器（Program Counter, PC）
存储当前正在执行指令的内存地址。

寄存器体系结构的演化

- **累加器（Accumulator）**
  - 来源于早期“加法机（adding machine）”时代的遗产。

- **累加器 + 地址寄存器（Accumulator + Address Registers）**
  - 需要通过寄存器间接寻址（register indirection）。
  - 起初，地址寄存器是专用寄存器（special-purpose），即只能加载一个地址，用于间接寻址。
  - 随后，地址运算（arithmetic on addresses）也得到了支持。

- **通用寄存器（General Purpose Registers, GPR）**
  - 所有寄存器都可用于各种用途。
  - 寄存器数量从早期的少量增加到：
    - 常见的 RISC 架构拥有 **32 个寄存器**；
    - Intel IA-64 体系结构中多达 **128 个寄存器**。

指令类别（Instruction Classes）

- **运算指令（Operate instructions）**
  - 处理数据：执行算术和逻辑运算。
  - 操作流程：取操作数（fetch operands）、计算结果（compute result）、存储结果（store result）。
  - 隐式顺序控制流（Implicit sequential control flow）：指令按顺序依次执行。

- **数据传输指令（Data movement instructions）**
  - 在内存、寄存器与 I/O 设备之间移动数据。
  - 隐式顺序控制流： 顺序执行数据移动操作。

- **控制流指令（Control flow instructions）**
  - 改变指令的执行顺序。
    （例如：跳转、分支、调用、返回等指令）

指令集架构（ISA）的组成元素

- **加载/存储（Load/Store） vs. 内存/内存（Memory/Memory）体系结构**

  - 加载/存储体系结构（Load/Store Architecture）
    - 运算指令只作用于寄存器。
    - 示例：MIPS、ARM 以及许多 RISC（精简指令集） 架构。

  - 内存/内存体系结构（Memory/Memory Architecture）
    - 运算指令可以直接作用于内存位置。
    - 示例：x86、VAX 以及许多 CISC（复杂指令集） 架构。

指令集架构（ISA）的组成元素

- 寻址模式（Addressing modes）用于指定如何获得操作数。
  - **立即数寻址**（Absolute）LW rt, 10000
    使用立即数（immediate value）作为地址。
  - 寄存器间接寻址（Register Indirect）LW rt, (r_base)
    使用寄存器 GPR[r_base] 的内容作为地址。
  - 带位移或基址寻址（Displaced or Based）LW rt, offset(r_base)
    使用 offset + GPR[r_base] 作为地址。
  - 索引寻址（Indexed）LW rt, (r_base, r_index)
    使用 GPR[r_base] + GPR[r_index] 作为地址。
  - 内存间接寻址（Memory Indirect）LW rt, ((r_base))
    使用 M[GPR[r_base]] 的值作为地址。
  - 自动递增/递减寻址（Auto Increment/Decrement）LW rt, (r_base)
    使用 GPR[r_base] 作为地址，但每次取数后会自动递增或递减 GPR[r_base]。

不同寻址方式的优点是什么？

- 又一个程序员与微体系结构设计师（microarchitect）之间的权衡例子

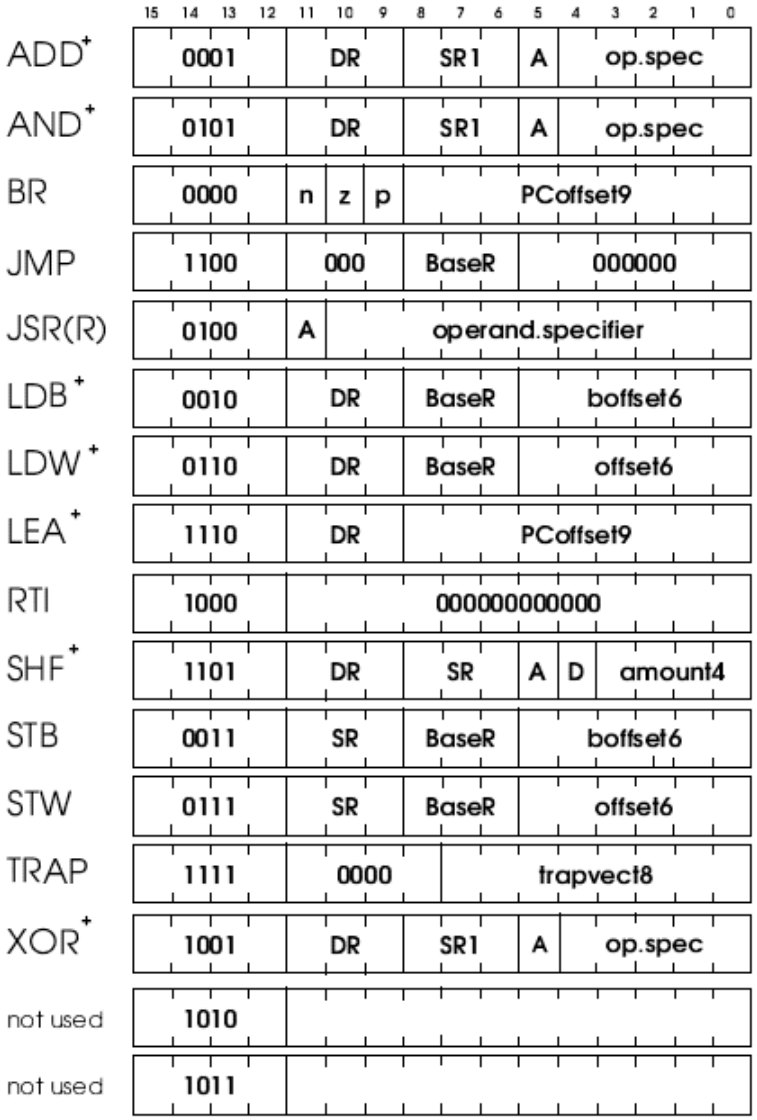
- 拥有更多寻址方式的优点：
  - 可以更好地将高级语言结构映射到机器层面
    一些数据访问用不同的寻址方式表达更自然，从而减少指令数量和代码体积。
    例如：
    - 数组访问（使用自动递增模式 *autoincrement mode*）
    - 间接寻址（指针追踪 *pointer chasing*）
    - 稀疏矩阵访问（*sparse matrix accesses*）
- 缺点：
  - 编译器需要做更多工作
  - 微体系结构设计师也需要做更多工作

ISA 正交性（ISA Orthogonality）

- **正交的 ISA（Orthogonal ISA）**
  - 所有寻址方式（addressing modes）都可以与所有指令类型（instruction types）组合使用。
  - **示例：VAX 架构**
    - 约 13 种寻址方式 × 超过 300 条操作码 × 整数与浮点格式（integer and FP formats）

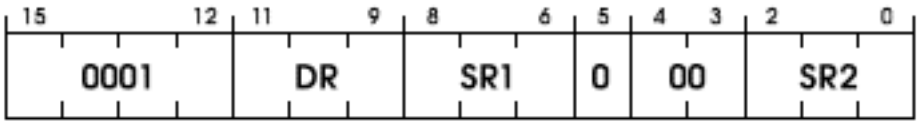
- **这对谁有好处？**
  ➤ 程序员（programmers）——因为他们拥有更大的灵活性，能用多种方式表达相同操作。
- **这对谁不利？**
  ➤ 微体系结构设计师（microarchitects）——因为硬件实现更复杂，解码和执行逻辑更难优化。

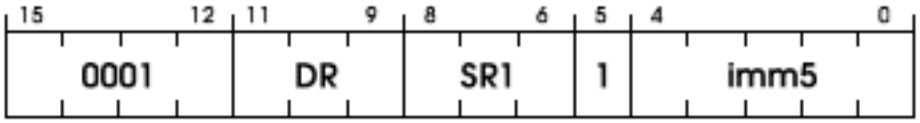
LC-3b 的 ISA 是正交的吗？



LC-3b：ADD 指令的寻址方式

Encodings



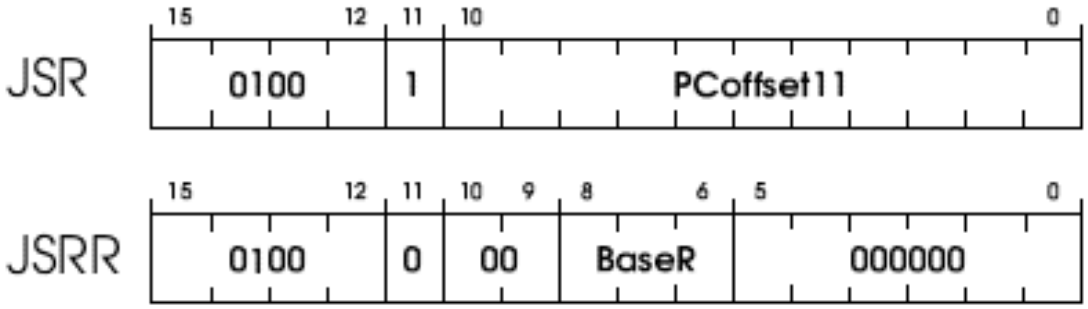


Operation

```
if (bit[5] == 0)
  DR = SR1 + SR2;
else
  DR = SR1 + SEXT(imm5);
setcc();
```

LC-3b：JSR(R) 指令的寻址方式

Encodings



Operation

```
R7 = PC† ;
if (bit[11] == 0)
  PC = BaseR;
else
  PC = PC† + LSHF(SEXT(PCoffset11), 1);
```

- **描述（Description）：** 首先，递增后的 PC（程序计数器） 被保存到 R7，用于从子程序返回。然后，处理器无条件跳转到子程序的起始地址，该地址由两种方式之一确定：当 bit[11] == 0 时：地址从寄存器 BaseR 中取得（即 JSRR 模式）。当 bit[11] == 1 时：地址通过 PC +（符号扩展的 PCoffset11 左移 1 位）计算得到（即 JSR 模式）。

指令集架构（ISA）的组成元素

- **如何与输入/输出（I/O）设备交互**
  - 内存映射 I/O（Memory-Mapped I/O）
    - 将一段特定的内存地址空间映射到 I/O 设备；
    - 对 I/O 设备的读写操作通过普通的**加载（load）和存储（store）**指令完成。
  - 专用 I/O 指令（Special I/O Instructions）
    - 某些体系结构（如 x86）提供专用的 I/O 指令；
    - 例如 IN 和 OUT 指令，用于通过端口（ports）与外设进行数据传输。
  - 权衡与取舍（Tradeoffs）
    - 哪种方式具备更强的通用性？
    - 内存映射 I/O 允许统一的寻址与访问机制，而专用 I/O 指令可以减少地址空间的占用。

指令集架构（ISA）的组成元素

- **特权模式（Privilege Modes）**
  - 用户态（User Mode）与管理态（Supervisor Mode）
  - 不同权限级别下，处理器限定哪些指令可以被执行
- **异常与中断处理（Exception and Interrupt Handling）**
  - 当指令执行过程中出现错误时，系统应遵循何种处理流程？
  - 当外部设备向处理器发出请求时，应如何响应？
  - 向量化中断（Vectored Interrupt）与非向量化中断（Non-vectored Interrupt）的区别（早期 MIPS 架构中的实现示例）
- **虚拟内存（Virtual Memory）**
  - 为每个程序提供一个“完整内存空间”的抽象，使其拥有比物理内存更大的地址空间幻觉
- **访问保护（Access Protection）**
  - 限制程序访问特定的内存区域或资源，以防止越权操作和系统破坏

再思考几个问题

- LC-3b 指令集体系结构（ISA）是否包含复杂指令？
- 一条指令的复杂程度可以达到什么水平？

复杂指令 vs. 简单指令

- 复杂指令（Complex Instruction）
  - 一条指令完成大量工作，即同时执行多个操作。
    - 在双向链表中插入元素（Insert in a doubly linked list）
    - 计算快速傅里叶变换（Compute FFT）
    - 字符串拷贝（String copy）
- 简单指令（Simple Instruction）
  - 一条指令仅执行少量工作，是构建复杂操作的基本单元（primitive）。
    - 加法（Add）
    - 异或（XOR）
    - 乘法（Multiply）

复杂指令 vs. 简单指令

- **复杂指令的优势**
  - 指令编码更密集（Denser encoding） → 代码体积更小（smaller code size） →
    提高内存利用率、节省片外带宽（off-chip bandwidth）、提升缓存命中率（cache hit rate），实现更高效的指令打包（better packing of instructions）。
  - 编译器更简单（Simpler compiler）：由于单条指令执行的功能更多，编译器无需对小粒度指令进行大量优化。

- **复杂指令的劣势**
  - 执行单元粒度较大（Larger chunks of work） → 编译器优化空间受限，难以进行精细化（fine-grained）优化。
  - 硬件设计更复杂（More complex hardware） → 需要在硬件层面完成从高级操作到控制信号的转换与优化，增加了设计与实现成本。

ISA 层级的权衡：语义鸿沟（Semantic Gap）

- **ISA 应处于何种层次？——语义鸿沟（Semantic Gap）**
  - 更接近高级语言（High-Level Language, HLL）
    → 语义鸿沟较小（small semantic gap），指令更复杂（complex instructions）更接近硬件控制信号（Hardware Control Signals）
    → 语义鸿沟较大（large semantic gap），指令更简单（simple instructions）
- **RISC 与 CISC 体系结构的比较**
  - RISC（Reduced Instruction Set Computer）
    精简指令集计算机，强调少而简单的指令、流水线化和编译器优化。
  - CISC（Complex Instruction Set Computer）
    复杂指令集计算机，提供功能丰富的指令以直接支持复杂运算。
    - FFT、快速排序（QUICKSORT）、多项式运算（POLY）、浮点运算指令（FP Instructions）
    - VAX 的 INDEX 指令：支持带边界检查的数组访问

ISA 层级的权衡：语义鸿沟（Semantic Gap）

- **编译器与硬件的复杂度取舍**
  - 简单编译器 + 复杂硬件 vs. 复杂编译器 + 简单硬件
    - 注意：指令翻译（translation）或间接层（indirection）可能改变这种权衡关系。
- **向后兼容性负担（Burden of Backward Compatibility）**
  - 为保持对旧 ISA 的兼容性，硬件设计常需保留冗余指令或微架构支持，增加实现复杂度。
- **性能与能耗（Performance & Energy Consumption）**
  - 优化责任分配：
    例如在 VAX 的 INDEX 指令中，优化应由编译器完成还是由硬件承担？
  - 相关考虑因素包括：
    - 指令长度（Instruction size）
    - 代码体积（Code size）

x86：小语义鸿沟的实例——字符串操作

- **字符串操作指令的特征**
  - 单条指令可作用于整个字符串：
    - 将任意长度的字符串从一个位置移动到另一个位置
    - 比较两个字符串

- **实现机制：重复执行能力（Repeated Execution in ISA）**
  - x86 ISA 支持通过“前缀（prefix）”机制实现指令重复执行
    - 使用 **REP 前缀（Repeat Prefix）** 指定重复操作次数

- **示例：REP MOVS 指令**
  - 指令长度仅为两个字节：
    - 一个 **REP 前缀字节** 与一个 **MOVS 操作码字节**（F2 A4）
  - 隐式源寄存器与目标寄存器：源地址寄存器：**ESI**，目标地址寄存器：**EDI**
  - 隐式计数寄存器**ECX** 指定字符串的长度

x86：小语义鸿沟的实例——字符串操作

REP MOVS (DEST SRC)

```
IF AddressSize = 16
    THEN
        Use CX for CountReg;
    ELSE IF AddressSize = 64 and REX.W used
        THEN Use RCX for CountReg; FI;
    ELSE
        Use ECX for CountReg;
FI;
WHILE CountReg ≠ 0
    DO
        Service pending interrupts (if any);
        Execute associated string instruction;
        CountReg ← (CountReg - 1);
        IF CountReg = 0
            THEN exit WHILE loop; FI;
        IF (Repeat prefix is REPZ or REPE) and (ZF = 0)
        or (Repeat prefix is REPNZ or REPNE) and (ZF = 1)
            THEN exit WHILE loop; FI;
    OD;
```

```
DEST ← SRC;
IF (Byte move)
    THEN IF DF = 0
        THEN
            (R|E)SI ← (R|E)SI + 1;
            (R|E)DI ← (R|E)DI + 1;
        ELSE
            (R|E)SI ← (R|E)SI - 1;
            (R|E)DI ← (R|E)DI - 1;
        FI;
    ELSE IF (Word move)
        THEN IF DF = 0
            (R|E)SI ← (R|E)SI + 2;
            (R|E)DI ← (R|E)DI + 2;
            FI;
        ELSE
            (R|E)SI ← (R|E)SI - 2;
            (R|E)DI ← (R|E)DI - 2;
        FI;
    ELSE IF (Doubleword move)
        THEN IF DF = 0
            (R|E)SI ← (R|E)SI + 4;
            (R|E)DI ← (R|E)DI + 4;
            FI;
        ELSE
            (R|E)SI ← (R|E)SI - 4;
            (R|E)DI ← (R|E)DI - 4;
        FI;
    ELSE IF (Quadword move)
        THEN IF DF = 0
            (R|E)SI ← (R|E)SI + 8;
            (R|E)DI ← (R|E)DI + 8;
            FI;
        ELSE
            (R|E)SI ← (R|E)SI - 8;
            (R|E)DI ← (R|E)DI - 8;
        FI;
FI;
```

小语义鸿沟 vs. 大语义鸿沟

- **CISC 与 RISC 的对比**
  - CISC（复杂指令集计算机） → 采用复杂指令
    - 产生背景：早期编译器生成的代码质量“不够好”，需要硬件通过复杂指令来弥补编译器能力的不足。
  - RISC（精简指令集计算机） → 采用简单指令
    - 代表人物：**John Cocke**（约1970年代中期，IBM 801 项目
    - **目标**：实现更好的编译器控制与优化能力



- **RISC 的设计动机**
  - 应对存储器等待（Memory Stalls）
    - 当复杂指令执行期间遇到存储器访问延迟时，处理器往往无法并行执行其他操作。
  - 简化硬件设计
    - 通过减少指令复杂度，降低设计成本，提高时钟频率。
  - 编译器优化（Compiler-Driven Optimization）
    - 编译器可更灵活地重排指令、挖掘细粒度并行性（fine-grained parallelism），以减少停顿（reduce stalls）。

可以高到什么程度？低到什么程度？

- **非常大的语义鸿沟（Very Large Semantic Gap）**
  - 每条指令直接指定机器中的完整控制信号集；
  - 编译器直接生成控制信号，而非抽象指令；
  - 代表思想：开放微码（Open Microcode）（John Cocke，约1970年代）；

- **非常小的语义鸿沟（Very Small Semantic Gap）**
  - 指令集体系结构（ISA）几乎等同于高级语言；
  - 示例：
    - **Java 虚拟机（Java Machines）**
    - **LISP 机器（LISP Machines）**
    - **面向对象机器（Object-Oriented Machines）**
    - **基于能力的机器（Capability-Based Machines）**

ISA 演化

- **指令集体系结构（ISA）的设计不断演化，以满足不同时代的技术约束与性能需求。**

- **典型历史背景**
  - 片上与片外存储容量受限（Limited on-chip and off-chip memory size）
  - 编译器优化能力有限（Limited compiler optimization technology）
  - 内存带宽受限（Limited memory bandwidth）
  - 特定应用领域的专用化需求（如多媒体扩展 MMX 指令）

- **翻译机制（Translation）的引入与作用**
  - 通过硬件（HW）或软件（SW）层的“指令翻译”，可在不同 ISA 间实现兼容或优化：
    - 无论上层 ISA 如何变化，底层实现可保持一致性。

指令翻译的作用（Effect of Translation）

- **可以通过将一种 ISA 翻译为另一种 ISA来调整语义鸿沟（semantic gap）之间的取舍。**
  - 例如：虚拟 ISA（Virtual ISA） ➜ 实际实现 ISA（Implementation ISA）。
  - Intel 与 AMD 的 x86 实现
    - 在硬件中将 x86 指令翻译为微操作（micro-operations, μops）
    - 这些微操作对程序员不可见，通常是更简单、更细粒度的内部指令。
  - Transmeta 的 x86 实现
    - 在软件层（Code Morphing Software）将 x86 指令翻译为“隐藏的”VLIW（超长指令字）指令。
- **翻译机制改变了传统 ISA 层的界限，使得同一 ISA 可拥有不同的实现策略。**
  - 设计者需权衡：
    - 性能与兼容性
    - 翻译层的复杂度与执行效率

基于硬件的指令翻译

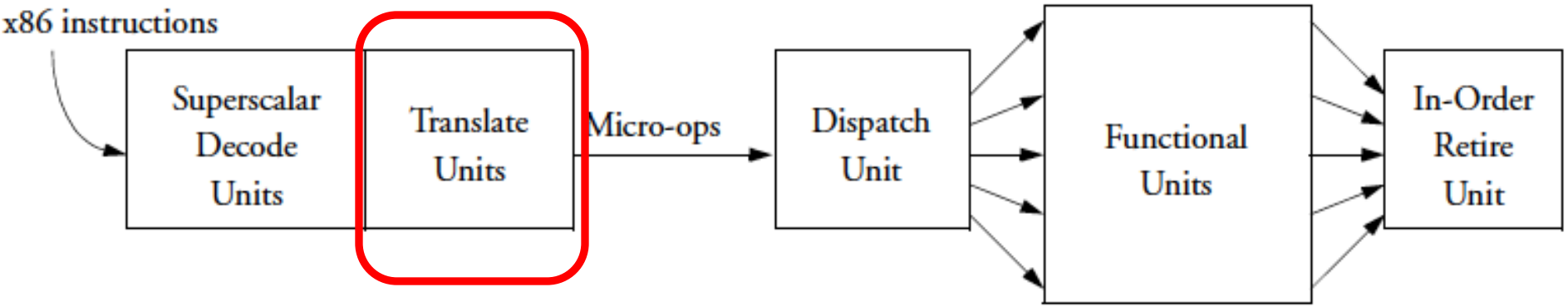


Figure 2. Conventional superscalar out-of-order CPUs use hardware to create and dispatch micro-ops that can execute in parallel.

Klaiber, "The Technology Behind Crusoe Processors," Transmeta White Paper 2000.

ISA 的权衡：指令长度

- 固定长度指令（Fixed Length）
  - 优点：
    - 硬件中解码单条指令较为容易。
    - 支持多条指令并行解码。
  - 缺点：
    - 指令中存在冗余位（Wasted bits），这会浪费存储空间。
    - 扩展 ISA（如何添加新指令）变得更难。
- 可变长度指令（Variable Length）
  - 优点：编码紧凑，有利于节省存储空间。
  - 缺点：
    - 解码单条指令需要更多的逻辑处理。
    - 不易于并行解码多条指令。
- 权衡（Tradeoffs）
  - 代码大小（内存空间、带宽、延迟）vs. 硬件复杂性
  - ISA 的可扩展性和表达能力 vs. 硬件复杂性
  - 性能 vs. 能效（小的代码 vs. 解码的简便性）